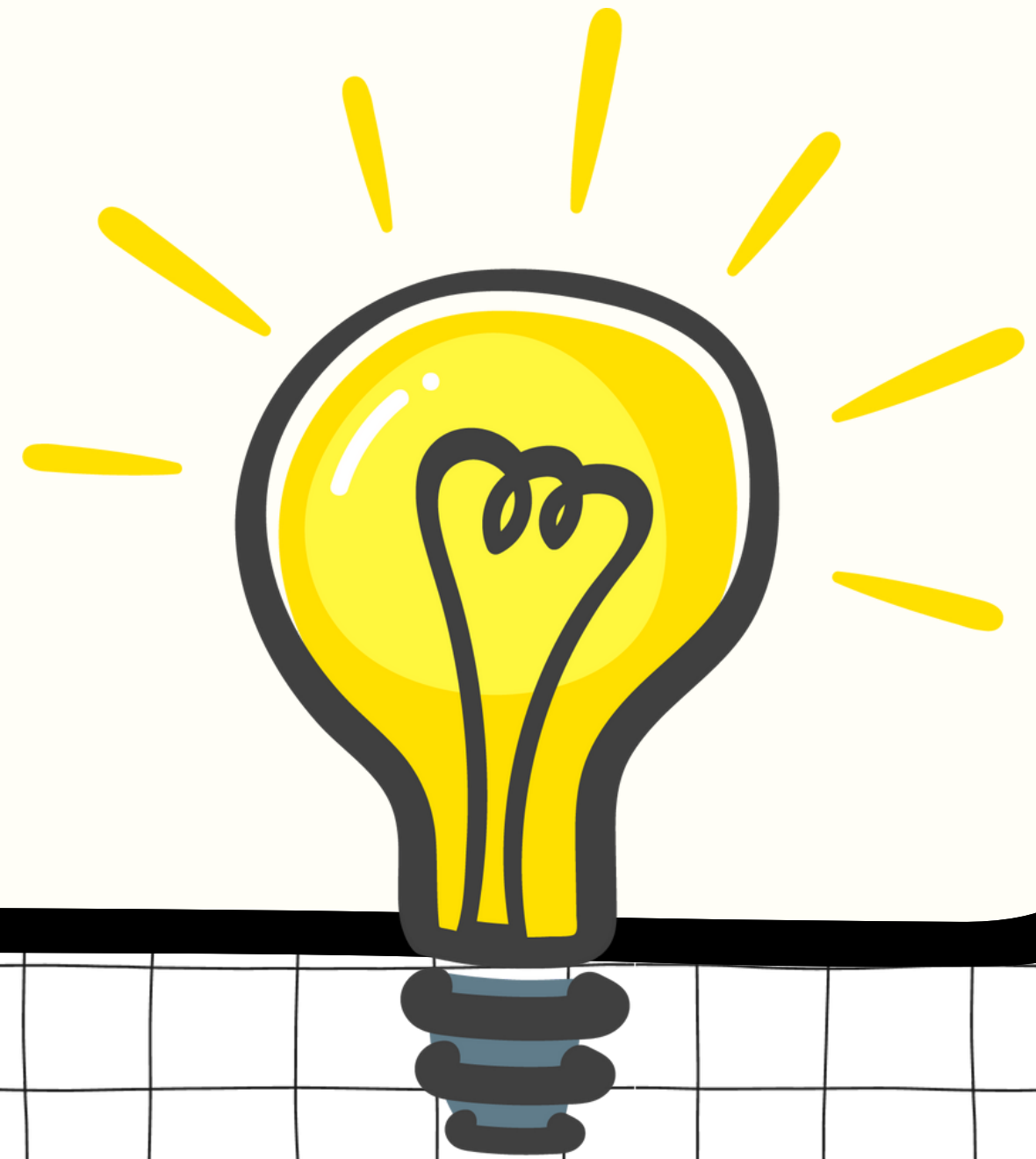


分析顧客類別與 預定房型之關係



童子瑄

資料收集與清理
錄製影片

陳妍蓁

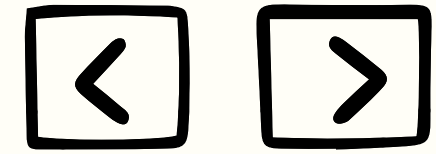
資料探索
上台報告

陳虹璇

機器學習
上台報告

全組成員皆有參與製作PPT與蒐集資料

Contents



1. Problem Definition

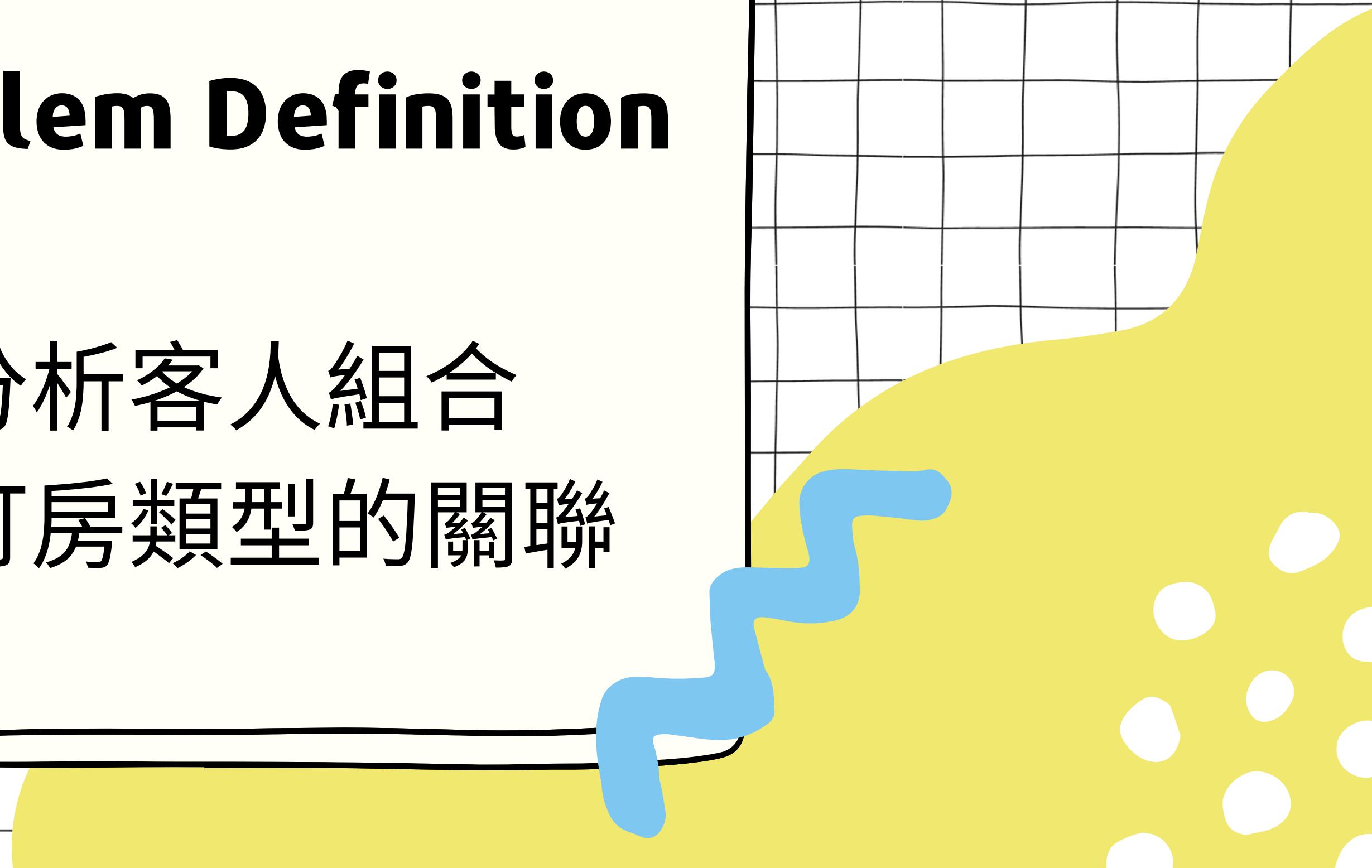
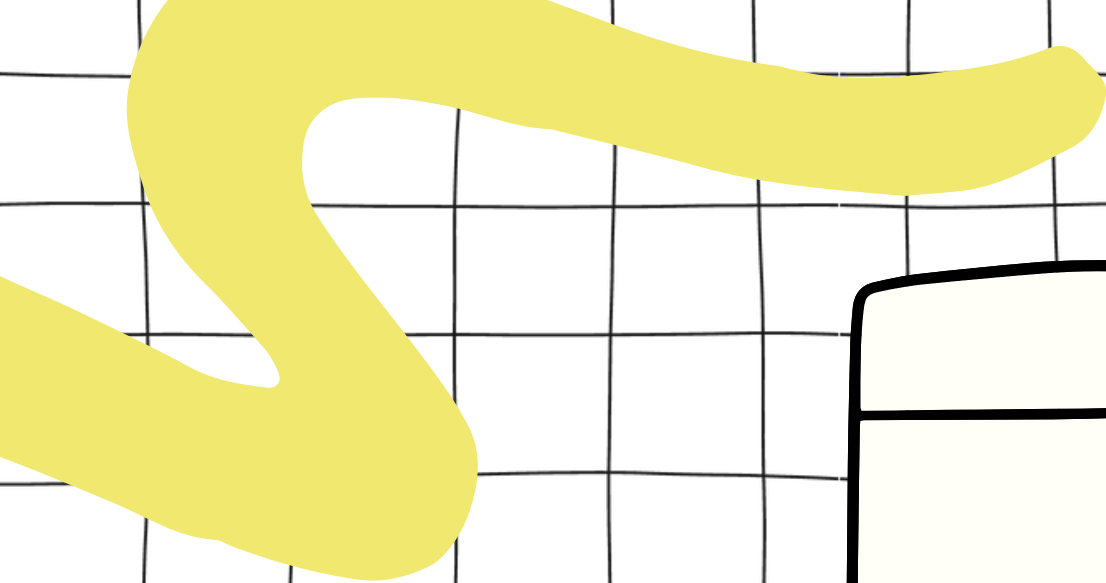
2. Data Collecting

3. Data Preparation

4. Data Exploratory

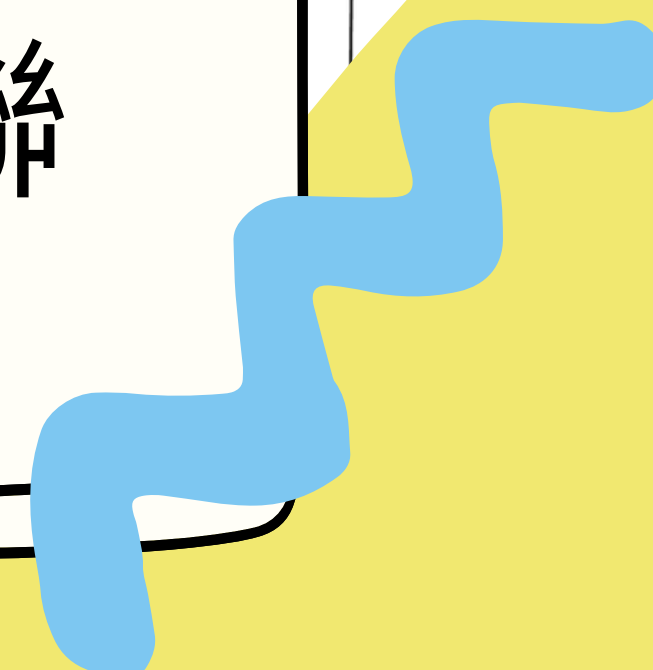
5. Model Building

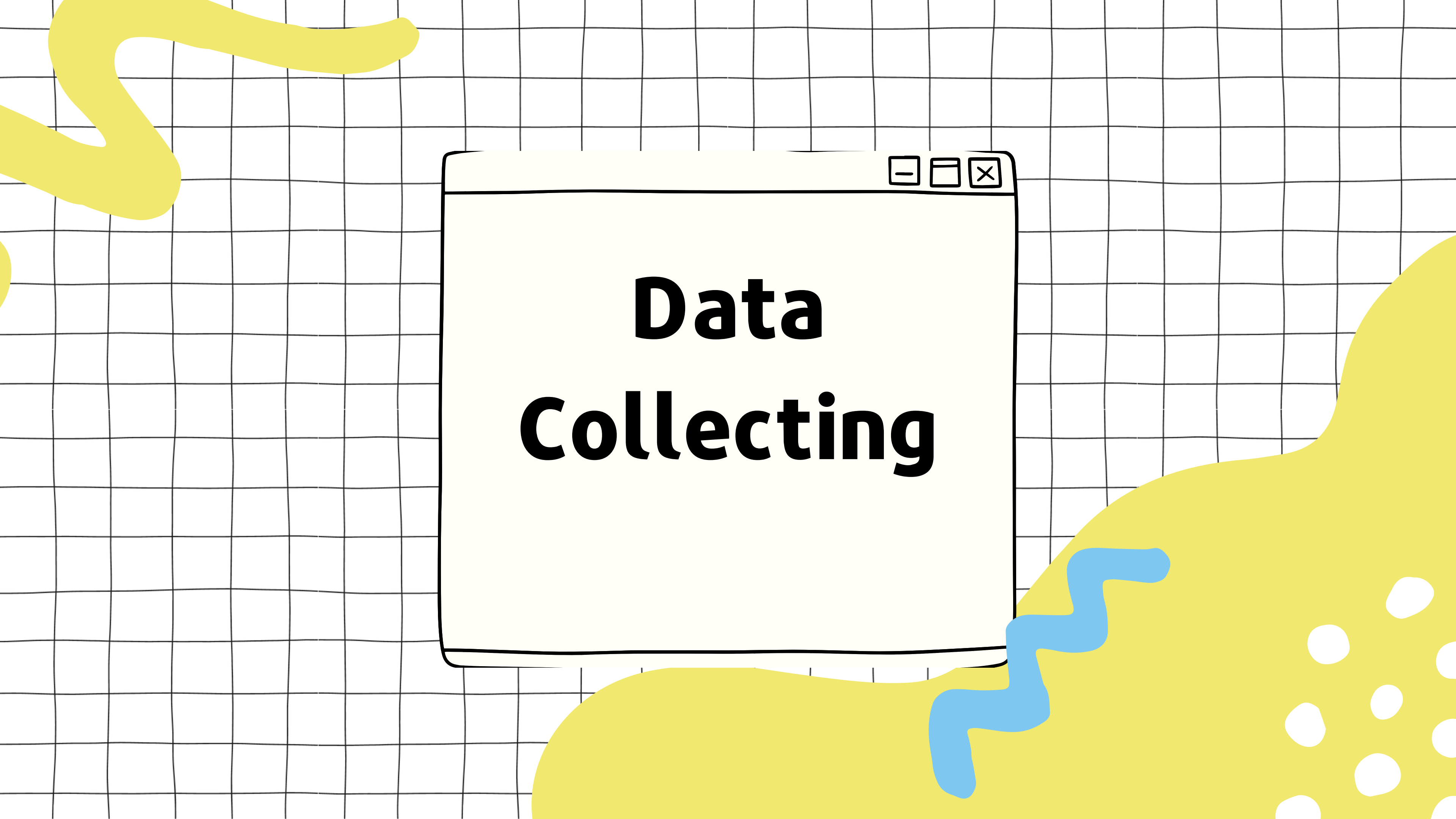
6. Data Presentation



Problem Definition

分析客人組合
與訂房類型的關聯





Data Collecting

在kaggle上挑選資料集並下載



Search



THE DEVASTATOR · UPDATED 8 DAYS AGO



26

New Notebook

Download (2 MB)



Hotel Bookings Analysis

Analyzing Hotel Bookings and Cancellations



Data Card

Code (4)

Discussion (1)

讀取"hotel_bookings.csv"文件中的數據

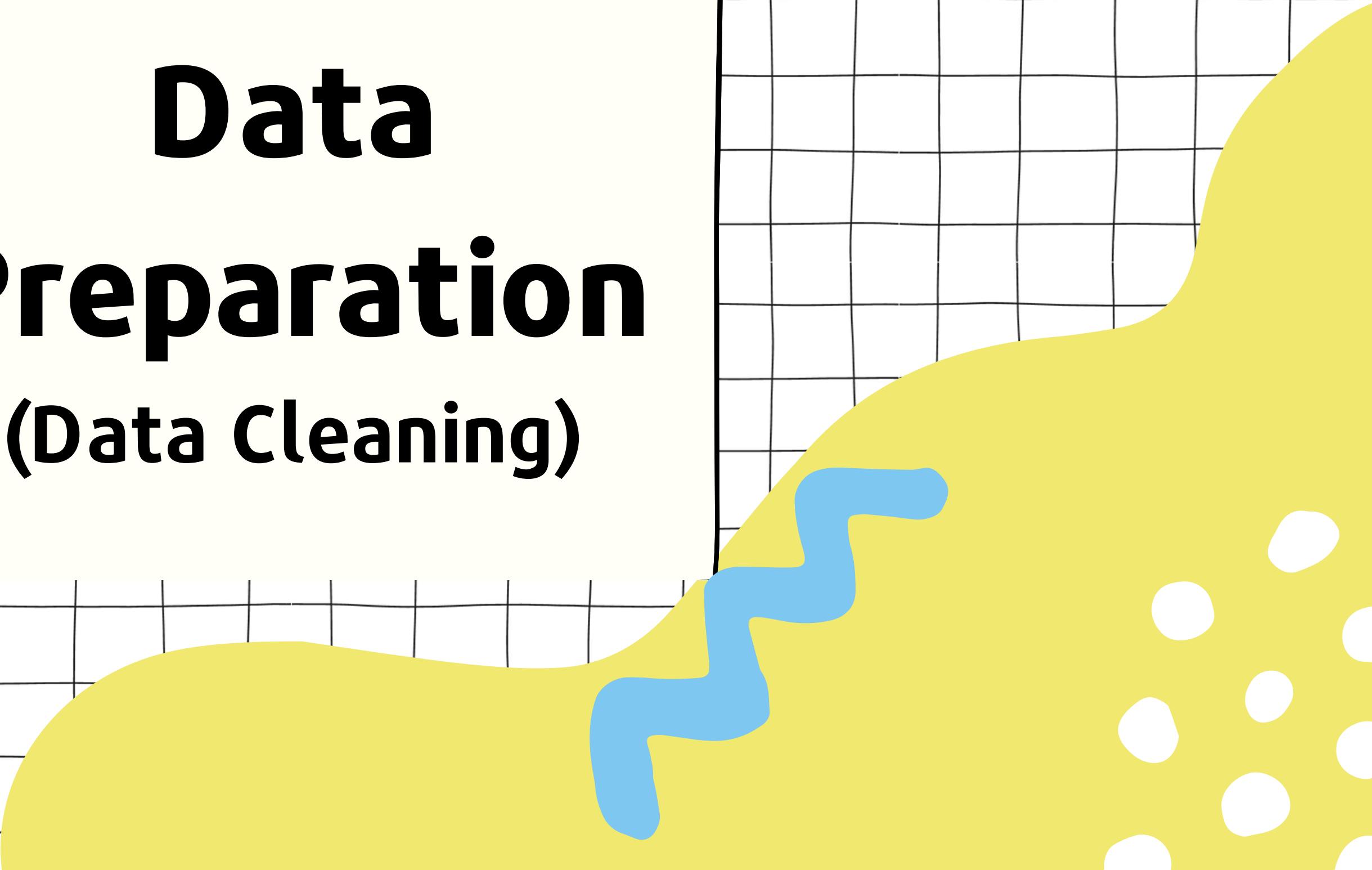
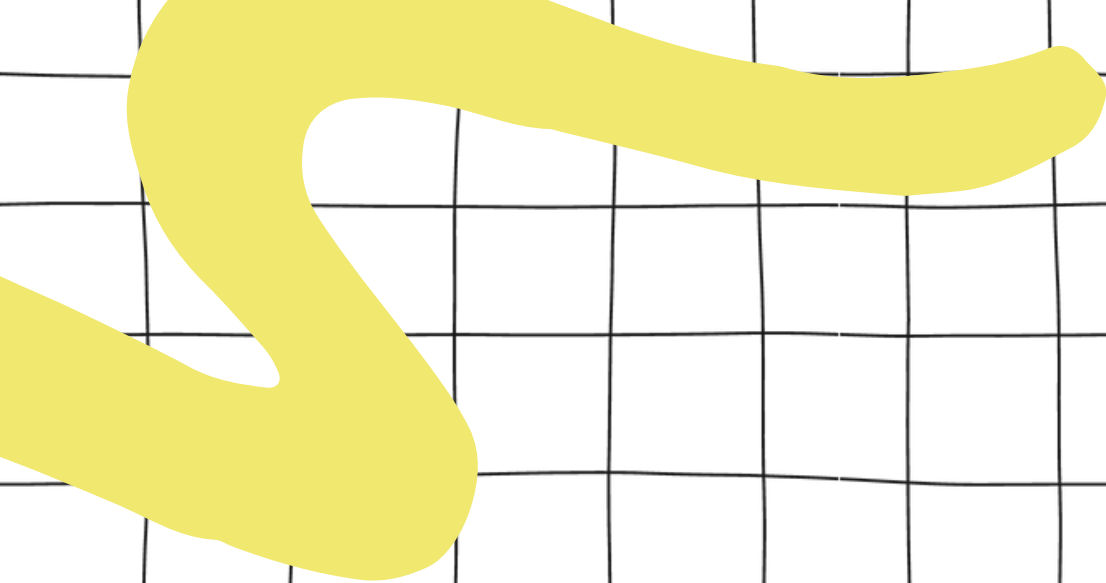
```
[ ] import pandas as pd
```

```
data = '/content/hotel_bookings.csv'  
df = pd.read_csv(data)  
df.head()
```

要先將csv
上傳至
colab

	index	hotel	is_canceled	lead_time	arrival_date_year	arrival_date
0	0	Resort Hotel	0	342	2015	
1	1	Resort Hotel	0	737	2015	
2	2	Resort Hotel	0	7	2015	
3	3	Resort Hotel	0	13	2015	
4	4	Resort Hotel	0	14	2015	

5 rows × 33 columns



Data Preparation (Data Cleaning)

移除不相關的資料欄位



```
[1] df = df.drop(columns = ['arrival_date_year', 'arrival_date_week_number', 'arrival_date_day_of_month', 'stays_in_weekend_nights',  
                        'stays_in_week_nights', 'hotel', 'is_canceled', 'lead_time', 'meal', 'reserved_room_type',  
                        'booking_changes', 'deposit_type', 'agent', 'company', 'days_in_waiting_list',  
                        'customer_type', 'adr', 'required_car_parking_spaces', 'country',  
                        'market_segment', 'distribution_channel', 'is_repeated_guest', 'previous_cancellations',  
                        'previous_bookings_not_canceled', 'total_of_special_requests', 'arrival_date_month', 'reservation_status',  
                        'reservation_status_date'])
```

```
df.head(n = 20)
```

	index	adults	children	babies	assigned_room_type
0	0	2	0.0	0	C
1	1	2	0.0	0	C
2	2	1	0.0	0	C
3	3	1	0.0	0	A
4	4	2	0.0	0	A
5	5	2	0.0	0	A
6	6	2	0.0	0	C
7	7	2	0.0	0	C
8	8	2	0.0	0	A

檢查是否有缺失值(未滿119390的數值皆為缺少數據)

刪除缺失值

```
[ ] df = df.dropna(subset = ['children'])  
df = df.reset_index(drop = True)  
df.head(n = 20)
```

	index	adults	children	babies	assigned_room_type
0	0	2	0.0	0	C
1	1	2	0.0	0	C
2	2	1	0.0	0	C
3	3	1	0.0	0	A
4	4	2	0.0	0	A
5	5	2	0.0	0	A
6	6	2	0.0	0	C
7	7	2	0.0	0	C

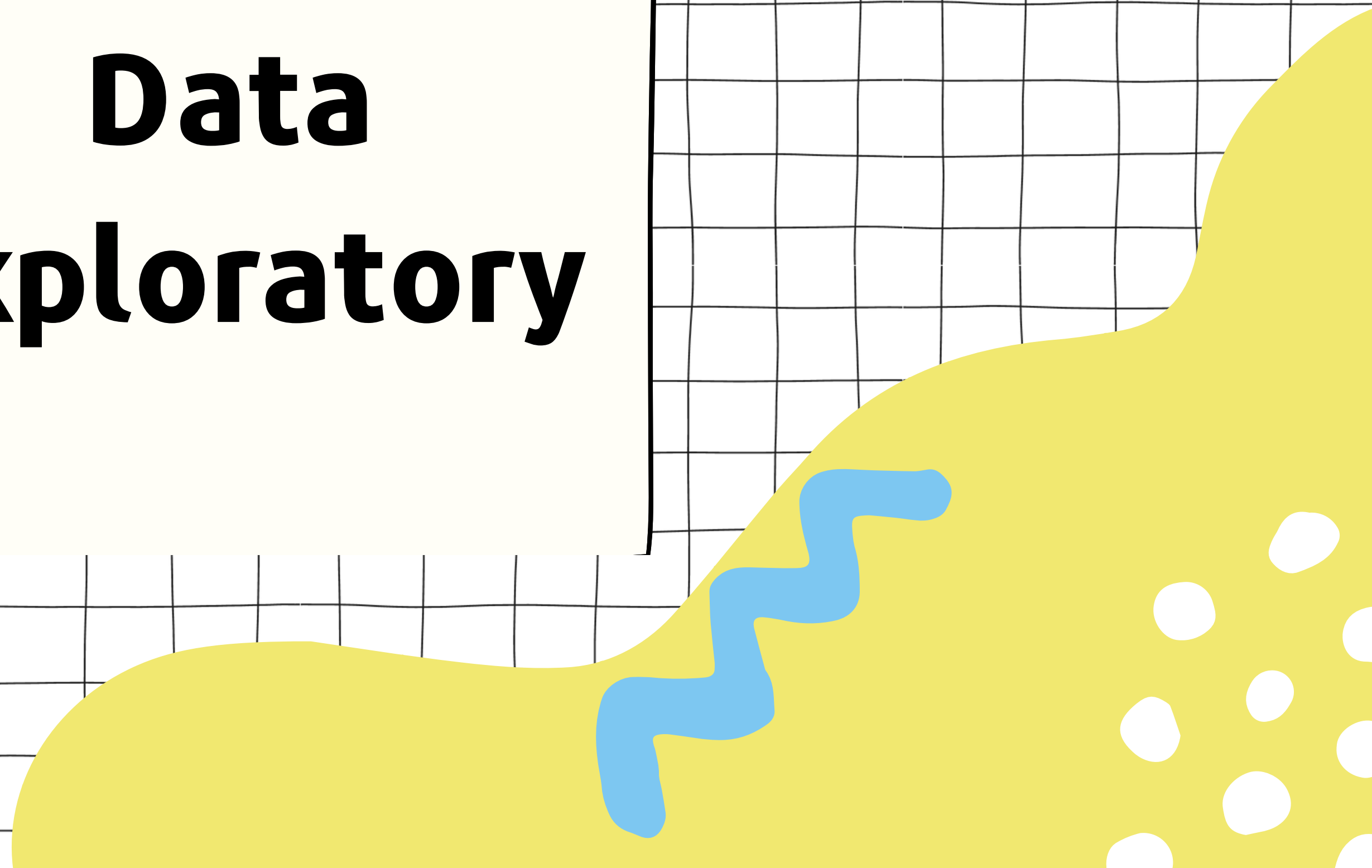
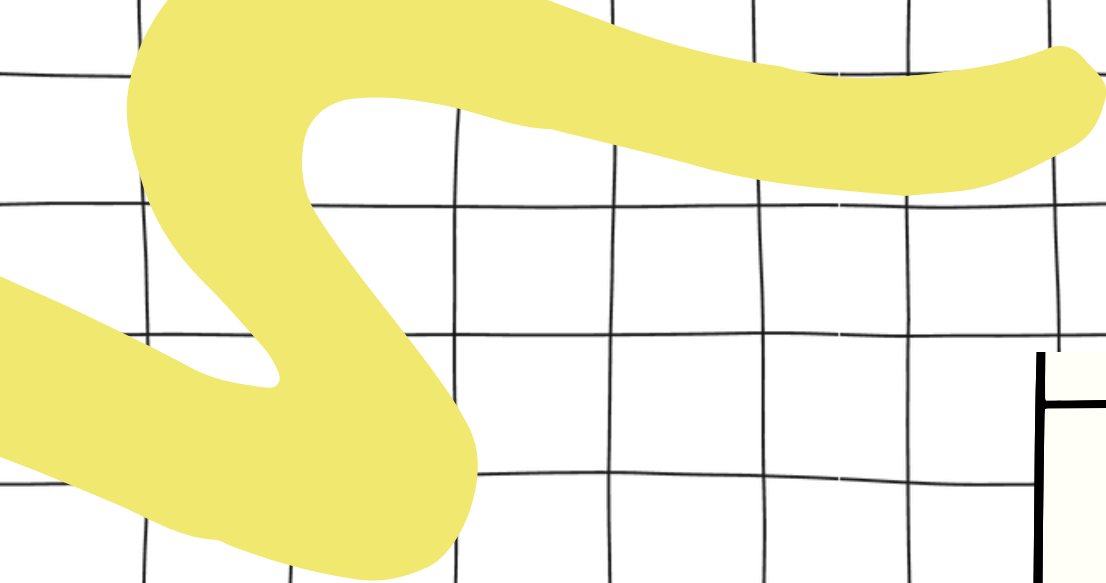
```
[ ] df.count()
```

```
index          119390  
adults         119390  
children       119386  
babies         119390  
assigned_room_type 119390  
dtype: int64
```

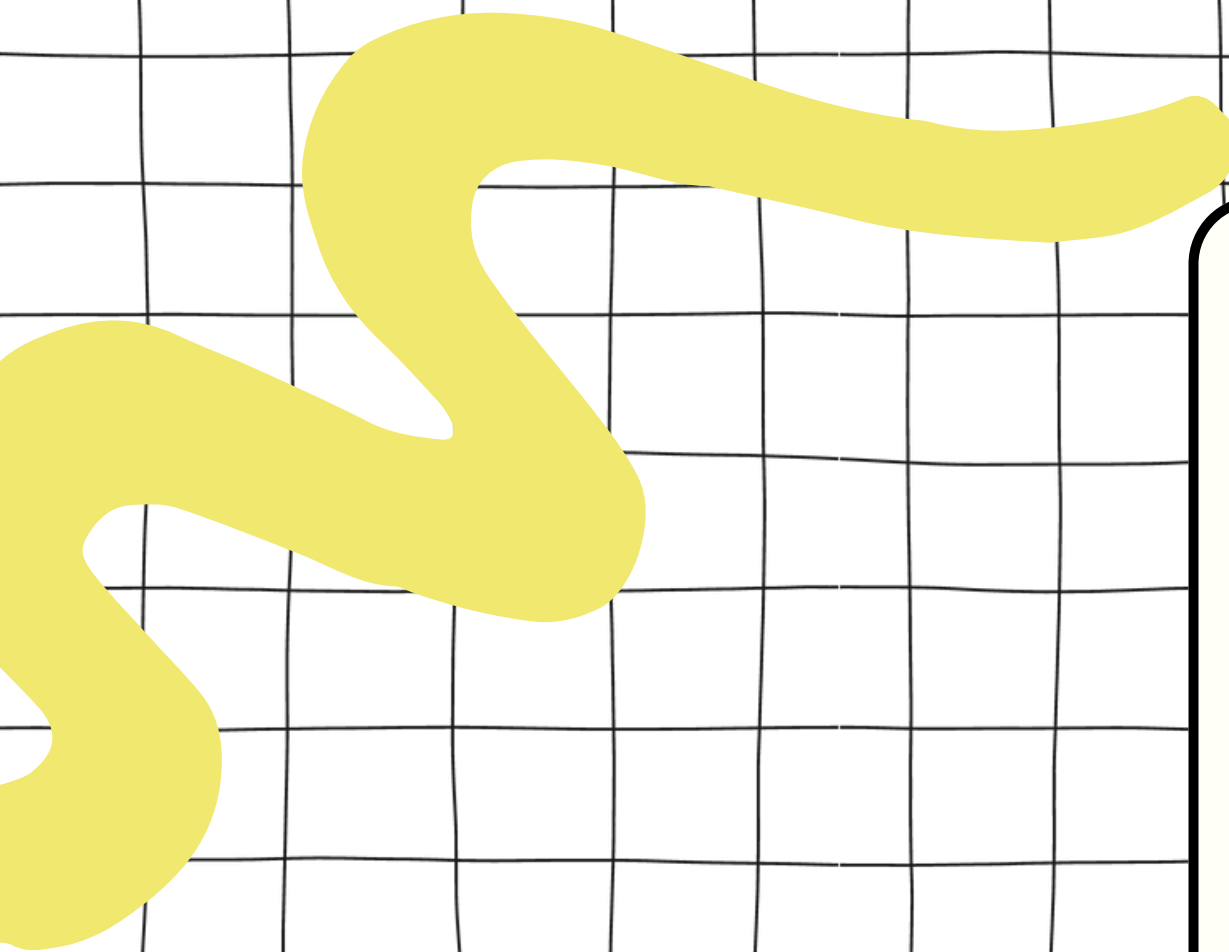
檢查是否有需要清理的重複項

```
[ ] df.nunique()
```

index	119386
adults	14
children	5
babies	5
assigned_room_type	12
dtype: int64	

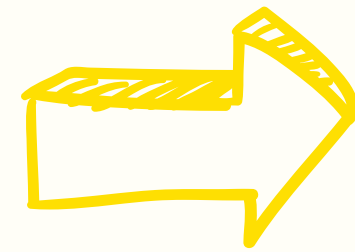


Data Exploratory

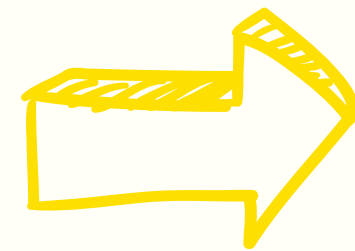


Data Exploratory

計算資料集的集中趨勢與離散程度

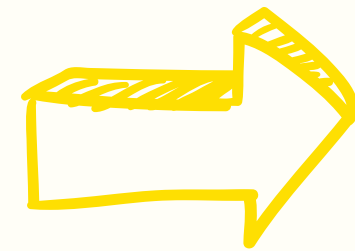


平均數、中位數、眾數

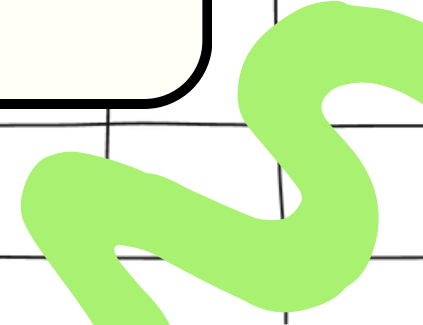


四分位距、變異數、標準差

以四分位距檢測異常值



檢測出異常值資料



Data Exploratory

眾數
平均值
中位數計算

```
▶ total_people_column = df["adults"]+df["children"]+df["babies"]

# 計算眾數
mode_value = total_people_column.mode()[0]

# 計算平均數
mean_value = total_people_column.mean()

# 計算中位數
median_value = total_people_column.median()

df = df.assign(total_people = total_people_column)

print(f"眾數: {mode_value:.2f}")
print(f"平均數: {mean_value:.2f}")
print(f"中位數: {median_value:.2f}")
df.head(n=20)
```



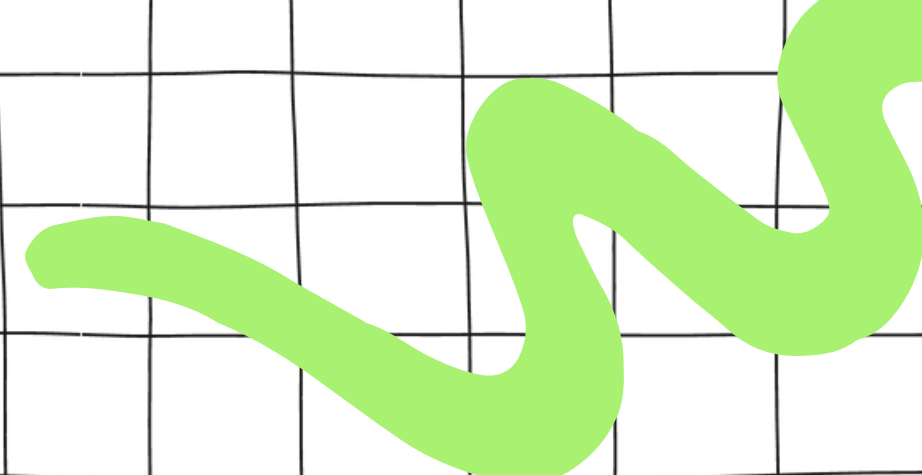
眾數 : 2.00
平均數 : 1.97
中位數 : 2.00

Data Exploratory

結果輸出

	index	adults	children	babies	assigned_room_type	total_people
0	0	2	0.0	0	C	2.0
1	1	2	0.0	0	C	2.0
2	2	1	0.0	0	C	1.0
3	3	1	0.0	0	A	1.0
4	4	2	0.0	0	A	2.0
5	5	2	0.0	0	A	2.0
6	6	2	0.0	0	C	2.0
7	7	2	0.0	0	C	2.0
8	8	2	0.0	0	A	2.0
9	9	2	0.0	0	D	2.0
10	10	2	0.0	0	E	2.0
11	11	2	0.0	0	D	2.0
12	12	2	0.0	0	E	2.0
13	13	2	1.0	0	G	3.0
14	14	2	0.0	0	E	2.0
15	15	2	0.0	0	E	2.0
16	16	2	0.0	0	E	2.0
17	17	2	0.0	0	E	2.0
18	18	2	0.0	0	G	2.0
19	19	2	0.0	0	G	2.0

Data Exploratory



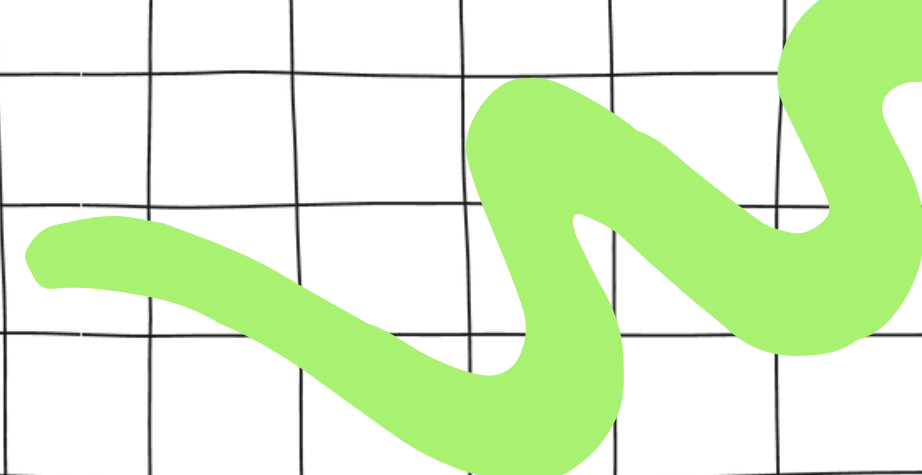
計算資料範圍
四分位數

```
[ ] #計算資料涵蓋範圍
print('Range')
print('({}, {})'.format(df.total_people.min(), df.total_people.max()))

#計算四分位數
print('Quartile')
print('Q1: {}'.format(df.total_people.quantile(q = 0.25)))
print('Q2: {}'.format(df.total_people.quantile(q = 0.5)))
print('Q3: {}'.format(df.total_people.quantile(q = 0.75)))
```

```
Range
(0.0, 55.0)
Quartile
Q1: 2.0
Q2: 2.0
Q3: 2.0
```

Data Exploratory



```
[ ] #計算變異數
    print('Variance')
    print(f"{df.total_people.var():.2f}")

    #計算標準差
    print('Standard deviation')
    print(f"{df.total_people.std():.2f}")
```

```
Variance
0.52
Standard deviation
0.72
```

計算變異數
標準差

Data Exploratory

以標準差
檢查異常值

```
upper_threshold = 0.52 + 3 * 0.72
lower_threshold = 0.52 - 3 * 0.72

df['is_outlier'] = (df['total_people'] > upper_threshold) | (df['total_people'] < lower_threshold)

display(df[df['is_outlier']])
```



	index	adults	children	babies	assigned_room_type	total_people	is_outlier
	13	2	1.0	0	G	3.0	True
	34	3	0.0	0	D	3.0	True
	35	3	0.0	0	D	3.0	True
	37	3	0.0	0	D	3.0	True
	38	3	0.0	0	D	3.0	True
...	...	...	...	...	...	...	...
119349	119353	3	0.0	0	D	3.0	True
119350	119354	3	0.0	0	D	3.0	True
119372	119376	3	0.0	0	E	3.0	True
119378	119382	3	0.0	0	G	3.0	True
119382	119386	3	0.0	0	E	3.0	True

14577 rows × 7 columns

Data Exploratory

以四分位距(IQR) 檢查異常值

```
[ ] q1 = df['total_people'].quantile(q=0.25)
    q3 = df['total_people'].quantile(q=0.75)
    iqr = q3 - q1

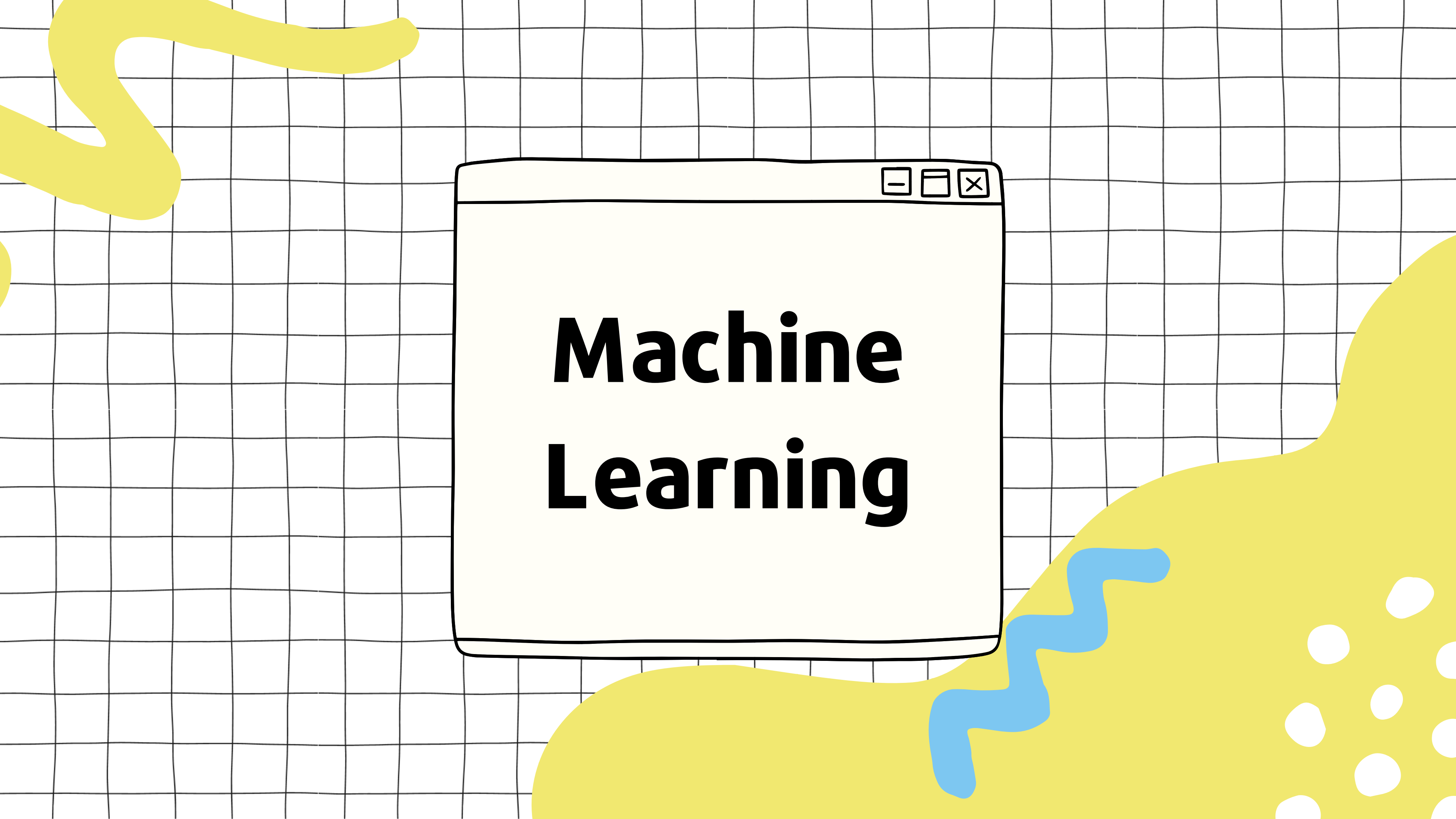
    def out_by_iqr(value, q1, q3, iqr):
        if value > q3 + 1.5 * iqr or value < q1 - 1.5 * iqr:
            return True
        else:
            return False

    outliers_series = df['total_people'].apply(out_by_iqr, args=(q1, q3, iqr))

    df = df.assign(out_by_iqr=outliers_series)

    display(df)
```

	index	adults	children	babies	assigned_room_type	total_people	out_by_iqr
0	0	2	0.0	0	C	2.0	False
1	1	2	0.0	0	C	2.0	False
2	2	1	0.0	0	C	1.0	True
3	3	1	0.0	0	A	1.0	True
4	4	2	0.0	0	A	2.0	False
...	...	...	...	...	...	...	...
119381	119385	2	0.0	0	A	2.0	False



Machine Learning

Machine Learning

1. 訓練、測試資料集

```
▶ from sklearn.model_selection import train_test_split

# 確保目標變數是 'assigned_room_type'，並將其轉換為類別型態
df['assigned_room_type'] = df['assigned_room_type'].astype('category')

# 分割資料集
train, test = train_test_split(df, test_size=0.2, random_state=42)

# 定義特徵和目標變數
features = ['adults', 'children', 'babies']
target = 'assigned_room_type'

train_X, train_y = train[features], train[target]
test_X, test_y = test[features], test[target]
```

目標：建立一個可以由客人類型組合預測並推薦房型的模型

Machine Learning

分類---決策樹

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

# 創建並訓練決策樹模型
dt = DecisionTreeClassifier(random_state=42).fit(train_X, train_y)

# 預測
predict_y = dt.predict(test_X)

# 評估模型性能
print("Confusion Matrix:")
print(confusion_matrix(test_y, predict_y))
print("\nAccuracy Score:", accuracy_score(test_y, predict_y))
```

```
➞ Confusion Matrix:
[[14628   5   1  165   0   5   0   0   0   4   0   0]
 [  384  33   0   11   0  14   1   0   0   2   0   0]
 [  326   0   5   52   2 104   1   3   0   1   0   0]
 [ 4282   0   2  720   0   4   0   2   0   0   0   0]
 [ 1394   0   6  114   1  23   2   6   0   1   0   0]
 [  457   0   4   42   0 264   4   5   0   0   0   0]
 [  206   0   7   33   0 232  23  18   0   0   0   0]
 [   41   0   0   14   0  12   8  68   0   0   0   0]
 [   63   0   0   0   0   6   0   1   0   1   0   0]
 [   38   0   0   1   0   0   0   2   0  20   0   0]
 [    1   0   0   0   0   0   0   0   0   0   0   0]
 [    0   0   0   0   0   0   0   0   0   3   0   0]]
```

Accuracy Score: 0.6601055364770919

Machine Learning

分類---隨機森林

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

# 創建並訓練隨機森林模型
rf = RandomForestClassifier(random_state=42).fit(train_X, train_y)

# 預測
predict_y = rf.predict(test_X)

# 評估模型性能
print("Confusion Matrix:")
print(confusion_matrix(test_y, predict_y))
print("\nAccuracy Score:", accuracy_score(test_y, predict_y))
```

```
➡ Confusion Matrix:
[[14628   5    1  165    0    5    0    0    0    4    0    0]
 [  384   33    0   11    0   14    1    0    0    2    0    0]
 [  326    0    5   52    0  106    1    3    0    1    0    0]
 [ 4282    0    2  720    0    4    0    2    0    0    0    0]
 [ 1394    0    6  114    1   23    2    6    0    1    0    0]
 [  457    0    4   42    0  264    4    5    0    0    0    0]
 [  206    0    7   33    0  232   23   18    0    0    0    0]
 [   41    0    0   14    0   12    8   68    0    0    0    0]
 [   63    0    0    0    0    6    0    1    0    1    0    0]
 [   38    0    0    1    0    0    0    2    0   20    0    0]
 [    1    0    0    0    0    0    0    0    0    0    0    0]
 [    0    0    0    0    0    0    0    0    0    3    0    0]]
```

Accuracy Score: 0.6601055364770919

Machine Learning

2. 測試模型

```
# 使用訓練好的決策樹模型進行預測
new_data = pd.DataFrame({'adults': [2], 'children': [1], 'babies': [0]})
prediction_dt = dt.predict(new_data)

# 使用訓練好的隨機森林模型進行預測
prediction_rf = rf.predict(new_data)

print("Decision Tree Prediction:", prediction_dt)
print("Random Forest Prediction:", prediction_rf)
```

```
➡ Decision Tree Prediction: ['A']
Random Forest Prediction: ['A']
```

Problems & Solve

在資料探索部分，我們根據老師的**PPT**來計算四分位距，但執行結果一直出現錯誤，原因為忘記將函數進行回傳

不清楚資料探索和資料準備之間的差別，所以花了一些時間去分清楚兩者的異同。



Thank you

